Credit Card Fraud Detection

Comparative evaluation of Logistic Regression, LightGBM, and XGBoost for imbalanced transaction classification.

AUTHOR
Viet Hung (Chris) Bui

GUIDED BY
Prof. Kevin Laughren

ABSTRACT

In this paper, we aim to identify a prediction model that can accurately detect credit card fraud at the lowest operating cost. To achieve this, the Credit Card Dataset retrieved from Worldline and the Machine Learning Group of ULB (Université Libre de Bruxelles), is divided into a training set (70%) and a testing set (30%), and apply 5-fold cross-validation tuning on three models—Logistic Regression, LightGBM, and XGBoost . This methodology repeatedly evaluates the models across different folds and selects the hyperparameter combination that produces the highest mean Area Under the ROC Curve score. Using these best-tuned parameters, we then evaluate them on the held-out test set across three key dimensions: Area Under the ROC Curve (AUC), Precision, Recall, F1 Score, and Confusion Matrix. In our cost framework, false alarms of a credit fraud are assumed to cost twice as much as non-fraud cases, while missed fraud cases cost five times more than correct non-fraud classifications. Throughout the tuning process, XGBoost is the best-performing model with an ability to identify 98% of fraud cases while keep the false-alarm of fraud to the minimum level at 66 cases. Therefore, XGBoost is selected for further analysis of feature importance to derive actionable business insights for the financial industry.

I. Introduction

Credit card fraud is a growing concern in the financial industry since fraudulent transactions causing significant financial losses each year, coupled with detecting fraud in real time is challenging due to the highly imbalanced nature of transaction data, where fraudulent activities comprise only a small fraction of all transactions.

In this paper, we analyze the **Credit Card Fraud Detection** dataset, originally released by **Worldline** and the **Machine Learning Group of ULB (Université Libre de Bruxelles)** as part of a research collaboration on big data mining and fraud detection, and made publicly available on Kaggle (*Dal Pozzolo et al., 2015*). The dataset contains approximately **285,000 transactions across 31 features**, including **492 fraudulent transactions**, which account for just **0.172%** of the total.

The objective of this study is to develop and compare machine learning models that can accurately predict fraudulent transactions (Class = 1) while minimizing false alarms (false positives) and missed fraud cases (false negatives). Specifically, we apply **Logistic Regression**, **Light Gradient Boosting (LightGBM)**, and **Extreme**

Gradient Boosting (XGBoost). Their performances are compared using the **Area Under the Receiver Operating Characteristic Curve (AUC)**, which visually measures a model's ability to distinguish between fraudulent and non-fraudulent transactions across all classification thresholds.

This research also ensures a fair comparison by evaluating each model in its best-performing state with different evaluation methods, rather than comparing an untuned baseline against finely tuned alternatives. This approach minimizes bias and provides a more reliable assessment of the true strengths and weaknesses of the models under optimal conditions

Lastly, each model's performance will be further evaluated using its **Confusion Matrix**, where **false alarms are assigned a cost twice as high as non-fraud cases**, while **missed fraud cases incur a cost five times greater**. This weighting is essential, as missing frauds generally have a greater financial impact than incorrectly flagging legitimate transactions. We therefore emphasize these costs to motivate minimizing both types of errors as much as possible. We also evaluate **Precision**, **Recall**, and the **F1 Score** to provide practical insights for fraud detection.

Precision reflects the proportion of transactions flagged as fraudulent that are actually fraud,

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall measures the proportion of actual frauds that the model successfully detects,

$$\text{Recall} = \frac{TP}{TP + FN}$$

and the **F1 Score** combines these two metrics into a balanced measure of performance.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

These metrics play an important role in evaluating highly imbalanced datasets, where overall accuracy can be misleading due to a model that predicts every transaction as non-fraudulent would still achieve over 99% accuracy but fail to identify any fraud.

1) Principal Component Analysis Approach

Due to the confidentiality of client information, the dataset has been transformed using **Principal Component Analysis (PCA)** to ensure that sensitive details are not exposed. This transformation serves to protect the privacy of transaction data while retaining the essential characteristics needed for fraud detection.

Principal Component Analysis (PCA) is a useful statistical technique that transforms the data into a set of new variables, while retaining the original data's variance and structure (*Pearson, 1901; Hotelling, 1933*). As long as the original dataset contains various features related to each transaction (e.g.time, merchant type,...), these variables can often be highly correlated, creating redundancy and unnecessary complexity. PCA addresses this by creating a new set of variables **V1 to V28**. These new variables are combinations of the original features,

designed to preserve the predictive power of the data. By applying PCA, we eliminate redundancy, enhance the efficiency of the machine learning models, and focus on the most important patterns related to fraudulent behavior, without exposing any confidential client information.

2) Necessary Packages and Data Set

PYTHON CELL 07

```python
#Import Necessary Functions
import warnings
warnings.filterwarnings("ignore")
import pandas as pd #Data wrangling & tabular data handling (similar to R's data.frame or dplyr).
import numpy as np  #Fast numerical computing.
import matplotlib
import matplotlib.pyplot as plt #Visualization & plotting (Python's version of R's plot).

from sklearn.model_selection import train_test_split #Package for splitting the data
from sklearn.model_selection import GridSearchCV     #Package for Cross Validation for Logistic Regression
from sklearn.model_selection import StratifiedKFold  #Package for Cross Validation for LightGBM and XGBoosts

from sklearn.metrics import roc_auc_score            #Function for ROC/AUC evaluation
from sklearn.metrics import roc_curve                #Funtion for plotting AUC curve
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import confusion_matrix         #Funtion for creating Confusion Matrix
from sklearn.metrics import precision_score          #Funtion for calculating Precision
from sklearn.metrics import recall_score             #Funtion for calculating Recall
from sklearn.metrics import f1_score                 #Funtion for calculating F1
from sklearn.metrics import classification_report    #Funtion for generating report (Precision, Recall, F1)

from sklearn.linear_model import LogisticRegression  #Funtion for generating Logistic Regression
from sklearn.preprocessing import StandardScaler     #Function for features scaling

import xgboost as xgb    #Function for generating XGBoost
import lightgbm as lgb   #Function for generating LightGBM
```

In this research, we work within JupyterLab, which offers an interactive environment that combines executable Python code with Markdown documentation in a structured and reproducible manner. This allows us to present both the implementation process and the reasoning behind it in a single workflow. The dataset is loaded into memory using the **pd.read_csv()** function and stored in the variable **CreditFraud**, which contains a collection of anonymized credit card transactions. This dataset serves as the foundation for identifying fraudulent activity and will be used consistently throughout the modeling and evaluation stages.

To ensure efficient data handling and analysis, we incorporate several core Python libraries. Specifically, **pandas** is employed for data wrangling and tabular manipulation (similar to R's data.frame structure), while **numpy** provides fast numerical computations on arrays and matrices. For visualization, we use **matplotlib**, which enables the creation of plots and graphs to help interpret model performance and dataset characteristics. Model development and evaluation are carried out using **scikit-learn**, a widely used machine learning library that provides tools for regression, classification, feature scaling, cross-validation, and performance metrics. Finally, to enhance predictive power through ensemble learning, we employ two state-of-the-art gradient boosting frameworks: **XGBoost** and **LightGBM**, both of which are highly optimized for handling large-scale and imbalanced datasets

II. Data Preprocessing

1) Data Splitting and Cleaning

PYTHON CELL 11

```python
def clean_dataframe(

    # Set up parameters for cleaning and splitting
    df: pd.DataFrame,
    target: str = "Class",
    test_size: float = 0.30,
    random_state: int = 2025,
    stratify: bool = True,):

    # Cleaning
    df = df.copy()
    df = df.drop_duplicates()
    df = df.dropna()
    df[target] = pd.to_numeric(df[target]).astype("int8")

    # Features/target split
    x = df.drop(columns=[target])
    y = df[target]

    # Train/test split
    x_train, x_test, y_train, y_test = train_test_split(
        x, y,
        test_size= test_size,
        random_state= random_state,
        stratify= y if stratify else None
    )

    return df, x_train, x_test, y_train, y_test

# Load data
CreditFraud = pd.read_csv("creditcard.csv")
df_clean, x_train, x_test, y_train, y_test = clean_dataframe(CreditFraud)

print("Training set size:", x_train.shape)
print("Testing set size:", x_test.shape)

print("Fraud ratio train:", round(y_train.mean(), 5))
print("Fraud ratio test :", round(y_test.mean(), 5))
```

OUTPUT

```
Training set size: (198608, 30)
Testing set size: (85118, 30)
Fraud ratio train: 0.00167
Fraud ratio test : 0.00167
```

Since our dataset contains 285,000 transactions, splitting the data is necessary to maintain sufficient observations in both training and testing sets while balancing model learning (train) and model evaluation (test). For this reason, we use a split of **70% for training** and **30% for testing**, which provides enough data for model training and ensures a reliable evaluation of performance. Additionally, we keep the fraud ratio consistent between the training and testing sets **(≈0.17%)** to ensure fair evaluation. By doing so, we guarantee that the model learns and is evaluated on distributions that accurately reflect real-world conditions.

A cleaning function is then applied to prepare the data. This function removes duplicate rows using **drop_duplicates()**, eliminates missing values with **dropna()**, and converts the target variable **Class** into an integer binary variable, where **0 indicates non-fraudulent transactions** and **1 represents fraudulent transactions**. These preprocessing steps ensure that the dataset is properly formatted and ready for training classification models.

2) Data Exploration

● PYTHON CELL 14

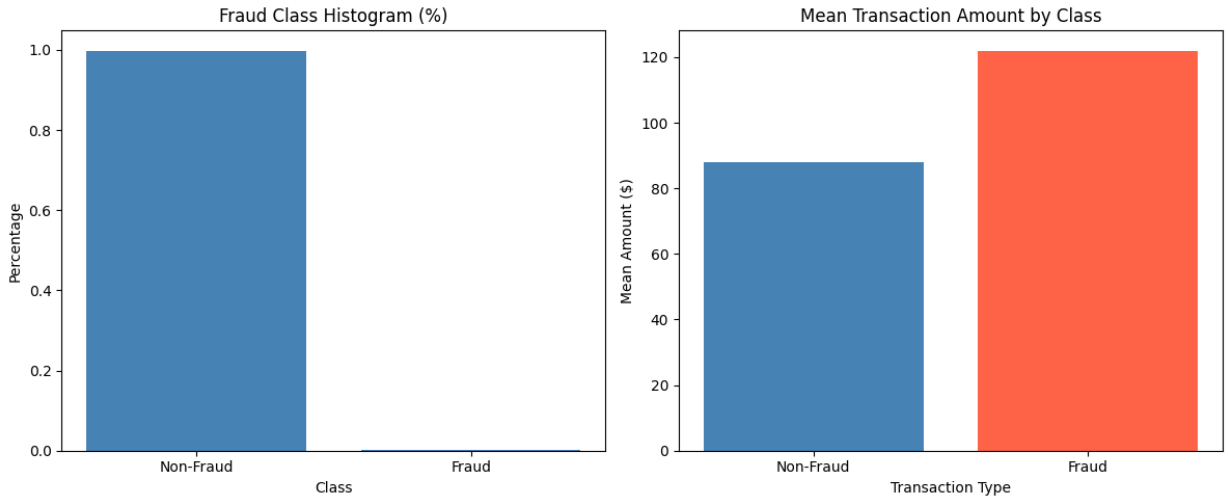
```python
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 5))  # 1 row, 2 columns

# Plot 1: Fraud Class Histogram
axes[0].bar(["Non-Fraud", "Fraud"], [0.9983, 0.0017], color="steelblue")
axes[0].set_title("Fraud Class Histogram (%)")
axes[0].set_ylabel("Percentage")
axes[0].set_xlabel("Class")

# Plot 2: Mean Transaction Amount by Class
axes[1].bar(["Non-Fraud", "Fraud"], [88, 122], color=["steelblue", "tomato"])
axes[1].set_title("Mean Transaction Amount by Class")
axes[1].set_ylabel("Mean Amount ($)")
axes[1].set_xlabel("Transaction Type")

plt.tight_layout()
plt.show()
```



We first examine the imbalance between fraudulent and non-fraudulent transactions in the dataset. As shown in the **Fraud Class Histogram**, fraud cases represent only about 0.17% of all transactions, while the vast majority are non-fraudulent. This extreme imbalance highlights the challenge for classification models, which can easily achieve high overall accuracy by predicting only the majority class but would fail to detect rare fraud cases.

Besides, the analysis of **Mean Transaction Amount by Class** also highlights the severity of credit card fraud. Although the average fraudulent transaction in the dataset is approximately €125, the cummulative impact of fraud is significant when scaled to real-world. For example, unauthorized credit card transactions are estimated to exceed **$6.2 billion** annually in the U.S. (Cruz, 2025). This problem is amplified by the fact that **80% of American adults hold at least one credit card**, and **60% have experienced unauthorized charges** (Cruz, 2025). Furthermore, **four out of five** victims report encountering fraud multiple times. These statistics underscore why it is crucial to evaluate models not just on overall accuracy, but on metrics that reflect their ability to detect rare and financially damaging fraud cases.

To begin our analysis, we examine the distribution of fraudulent versus non-fraudulent transactions in the dataset. As illustrated in the **Fraud Class Histogram**, fraudulent cases account for only about **0.17%** of all transactions, while the majority are non-fraudulent. This extreme class imbalance presents a fundamental challenge for machine learning models: a classifier could achieve over 99% accuracy simply by predicting every transaction as non-fraudulent, yet such a model would be practically useless because it fails to detect the rare but financially damaging fraudulent cases. Thus, overall accuracy alone is not a sufficient metric for evaluating model performance in this context.

In addition, the **Mean Transaction Amount by Class** chart highlights the severity of fraudulent activity. On average, **fraudulent transactions** in this dataset amount to **approximately €125**, compared to an average of **€88 for non-fraudulent** ones. While the numerical difference may appear modest at the individual level, its impact becomes highly significant when scaled to millions of transactions processed daily in the financial sector. For example, unauthorized credit card transactions in the United States alone are projected to exceed $6.2 billion annually (*Cruz, 2025*). The magnitude of the problem is further underscored by the fact that 80% of American adults own at least one credit card, and 60% have reported experiencing unauthorized charges, while four out of five fraud victims encounter fraudulent activity multiple times throughout their lifetime (*Cruz, 2025*). These statistics emphasize the real-world stakes of fraud detection and reinforce the importance of developing models that go beyond overall accuracy to prioritize metrics such as Recall, Precision, and F1 Score, which better capture the ability to identify rare fraudulent cases.

III. Fitting Models

1) Logistic Regression

● PYTHON CELL 17

```python
logit = LogisticRegression(max_iter=2000, random_state=2025)

param_grid = {
    "penalty": ["l1", "l2"],            #L1 is Ridge, L2 is Lasso
    "C": [0.01, 0.1, 1, 10, 100],
    "solver": ["liblinear"],            # works with l1 and l2
    "class_weight": [None, "balanced"]
}

grid = GridSearchCV(
    estimator=logit,
    param_grid=param_grid,
    scoring="roc_auc",   # AUC is good for imbalanced data
    cv=5,
    n_jobs=-1
)

grid.fit(x_train, y_train)


# Build CV results DataFrame
cv = pd.DataFrame(grid.cv_results_)
cv = cv[["mean_test_score","std_test_score","param_penalty","param_C","param_class_weight"]].copy()
cv["param_C"] = cv["param_C"].astype(float)
```

For Logistic Regression model, we first apply a grid search with **5-fold cross-validation**. The parameter grid explores the settings of the regularization penalty, **Lasso (L1)** and **Ridge (L2)**, the **regularization strength** (C values ranging from 0.01 to 100), and the use of **class weights** (None vs. balanced) to address class imbalance.

The purpose of this tuning is due to penalty shrinkage improves the robustness of Logistic Regression in the highly imbalanced fraud dataset. The **Ridge (L2) penalty** stabilizes coefficient estimates by shrinking them toward zero, which is crucial when predictors are correlated, as in the PCA-transformed components. In contrast, the **Lasso (L1) penalty** goes further by shrinking some coefficients exactly to zero, effectively performing feature selection and simplifying the model. By tuning across both Ridge and Lasso penalties, we ensure that Logistic Regression is evaluated under conditions that balance stability and sparsity. This reduces overfitting and improves generalization to unseen transactions, making the model more reliable for fraud detection.

Because the dataset is highly imbalanced, with fraud cases representing less than 0.2% of all transactions, we apply the **balanced** option for **class weights**. This setting automatically adjusts weights in inverse proportion to class frequencies. Particularly, each fraudulent case is treated as roughly **578 times** more important than a non-fraudulent case, hence, the model faces a much stronger penalty for misclassifying fraud than for misclassifying a legitimate transaction.

$$weight_j = \frac{n_{\text{samples}}}{n_{\text{classes}} \times n_j}$$

For fraud:

$$w_{\text{fraud}} = \frac{285000}{2 \times 492} \approx 289.63$$

For non-fraud:

$$w_{\text{nonfraud}} = \frac{285000}{2 \times 284508} \approx 0.50$$

So the ratio is:

$$\frac{w_{\text{fraud}}}{w_{\text{nonfraud}}} \approx 578$$

This adjustment plays a crucial role in preventing the model from defaulting to the solution of labeling all transactions as non-fraudulent, which would achieve high accuracy but fail in real-world detection. Additionally, it ensures the model gives attention to the minority class, effectively "forcing" the model to recognize and detect fraud cases despite their rarity. Consequently, reweighting is an essential step when handling imbalance financial data due to the loss from overlooking a fraudulent transaction is greater than the inconvenience of incorrectly flagging a valid one. By weighting fraud cases more heavily, the training process is aligned with real-world priorities, focusing not just on overall accuracy but on reducing costly undetected fraud.

The **solver** is restricted to **liblinear**, which supports both L1 and L2 penalties. Each parameter combination is evaluated using AUC as the scoring metric, since AUC is particularly appropriate for highly imbalanced datasets like fraud detection. Finally, we store cross-validation results in a DataFrame, summarizing the mean and standard deviation of AUC scores for each parameter set.

PYTHON CELL 19

```python
def pivot_for(cw):
    sub = cv[cv["param_class_weight"] == cw]
    tbl = sub.pivot(index="param_penalty", columns="param_C", values="mean_test_score")
    # ensure consistent ordering
    return tbl.reindex(index=["l1","l2"]).reindex(columns=sorted(tbl.columns))
```

```
# Only keep the balanced table
tbl_bal = pivot_for("balanced")

fig, ax = plt.subplots(figsize=(6, 4))

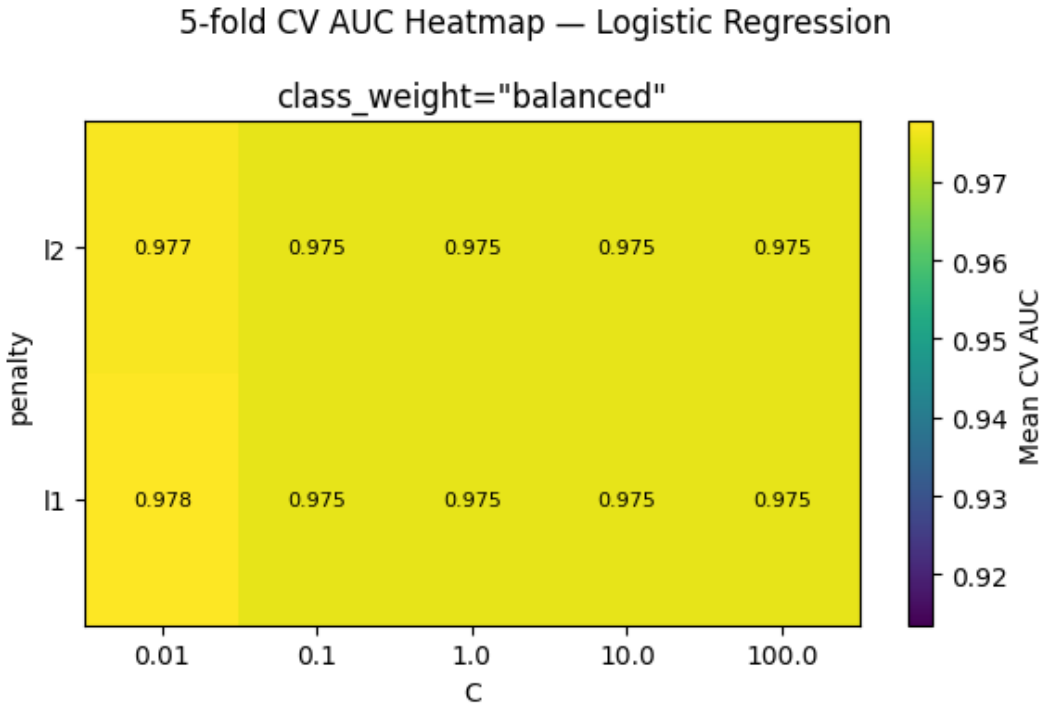
im = ax.imshow(tbl_bal.values, aspect="auto", origin="lower", cmap="viridis",
               vmin=cv["mean_test_score"].min(), vmax=cv["mean_test_score"].max())

# ticks & labels
ax.set_xticks(np.arange(len(tbl_bal.columns)))
ax.set_xticklabels([str(c) for c in tbl_bal.columns], rotation=0)
ax.set_yticks(np.arange(len(tbl_bal.index)))
ax.set_yticklabels(tbl_bal.index)
ax.set_xlabel("C")
ax.set_ylabel("penalty")
ax.set_title('class_weight="balanced"')

# annotate cells
for i in range(tbl_bal.shape[0]):
    for j in range(tbl_bal.shape[1]):
        ax.text(j, i, f"{tbl_bal.values[i, j]:.3f}", ha="center", va="center", fontsize=8)

# colorbar
fig.colorbar(im, ax=ax, label="Mean CV AUC")
plt.suptitle("5-fold CV AUC Heatmap — Logistic Regression")
plt.tight_layout()
plt.show()

# Best params from GridSearchCV
best_logit = grid.best_estimator_
print("Best parameters:", grid.best_params_)
print(f"Best CV AUC: {grid.best_score_:.4f}")
```



OUTPUT

```
Best parameters: {'C': 0.01, 'class_weight': 'balanced', 'penalty': 'l1', 'solver': 'liblinear'}
Best CV AUC: 0.9776
```

We then visualize the impact of hyperparameter tuning on Logistic Regression by creating a heatmap of the cross-validation results. The function **pivot_for** restructures the grid search output to show mean cross-validated AUC scores across different values of the **regularization strength (C)** and **penalty type (L1 or L2)**, filtered for the case where class weights are set to **balanced**. The heatmap uses color intensity to represent model performance, with numerical AUC values annotated in each cell for clarity.

The cross-validation heatmap for Logistic Regression demonstrates that model performance remains consistently high across different settings of the penalty type (L1 vs. L2) and regularization strength (C). All parameter combinations yield **mean AUC** values **between 0.975 and 0.978**, indicating that Logistic Regression is highly stable under balanced class weighting. The best performance is observed at **C=0.01** with a **Lasso (L1) penalty**, producing a **mean CV AUC of 0.9776**. The small differences across C values suggest that stronger regularization (smaller C) slightly improves generalization, while higher C values offer no additional benefit and may risk overfitting. The similarity between Lasso (L1) and Ridge (L2) penalties also suggests that both approaches handle the dataset effectively, though L1 may provide slightly finer advantage of feature selection. Overall, these results highlight that logistic regression, when properly tuned with balanced class weights, provides a reliable baseline for fraud detection, and achieve a strong mean AUC score.

PYTHON CELL 21

```python
# Logistic Regression
best_logit.fit(x_train, y_train)

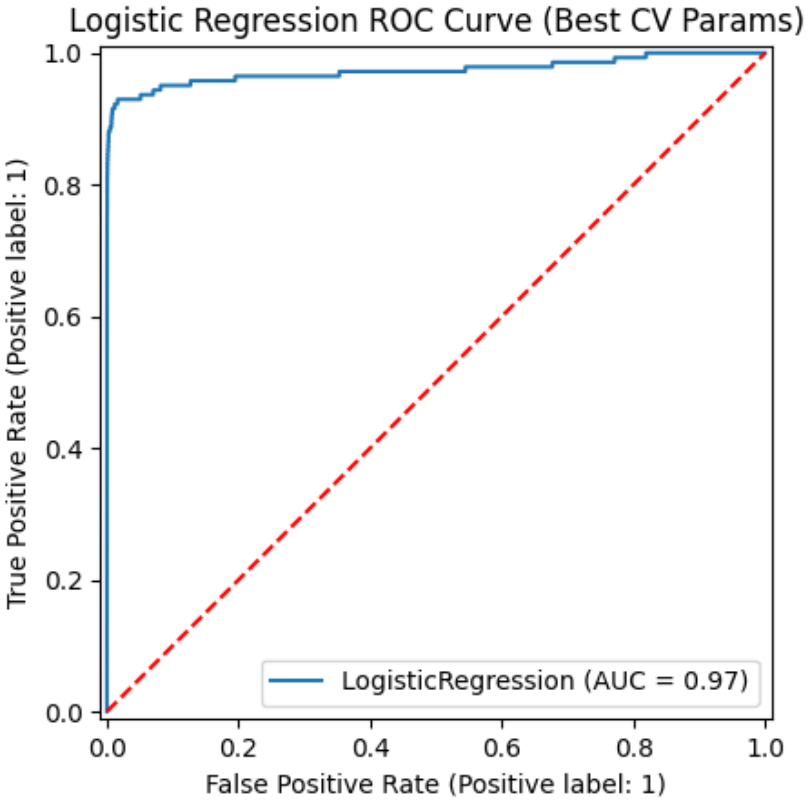
y_prob_test = best_logit.predict_proba(x_test)[:, 1]

threshold = 0.9
y_pred_test = (y_prob_test >= threshold).astype(int)

# Precision, Recall, F1, AUC
precision = precision_score(y_test, y_pred_test, zero_division=0)
recall    = recall_score(y_test, y_pred_test, zero_division=0)
f1        = f1_score(y_test, y_pred_test, zero_division=0)
auc       = roc_auc_score(y_test, y_prob_test)

# ROC curve on test
RocCurveDisplay.from_estimator(best_logit, x_test, y_test)
plt.plot([0,1],[0,1],"r--")
plt.title("Logistic Regression ROC Curve (Best CV Params)")
plt.show()

print(f"AUC (prob-based): {auc:.4f}")
print("\nClassification report:")
print(classification_report(y_test, y_pred_test, digits=4))
```



OUTPUT

```
AUC (prob-based): 0.9733

Classification report:
              precision    recall  f1-score   support

           0     0.9998    0.9931    0.9964     84976
           1     0.1774    0.8944    0.2960       142

    accuracy                         0.9929     85118
   macro avg     0.5886    0.9437    0.6462     85118
weighted avg     0.9985    0.9929    0.9953     85118
```

Using the cross-validated best hyperparameters for **Logistic Regression (C = 0.01, penalty = L1, solver = liblinear, class_weight = balanced)**, we refit the model on the full training set and evaluated it on the held-out test set, which contains 85,118 observations with 142 fraud cases. The model achieved an **AUC of 0.9733**, which is considered an outstanding result and demonstrates strong ability to rank fraudulent transactions above non-fraudulent ones. However, AUC alone does not capture the trade-off between false positives and false negatives at a specific threshold. Therefore, it is important to also evaluate Precision, Recall, and F1-score, which provide deeper insights into the model's practical performance in detecting rare fraud cases.

● PYTHON CELL 23

```python
# Confusion matrix + metrics
cm = confusion_matrix(y_test, y_pred_test)
tn, fp, fn, tp = cm.ravel()
total = cm.sum()

# Cost model: FP costs 2, FN costs 5
cost_fp = 2
cost_fn = 5
total_cost = cost_fp*fp + cost_fn*fn

# Confusion Matrix
```

```
labels = np.array([["TN", "FP"], ["FN", "TP"]])
fig, ax = plt.subplots(figsize=(6.5, 5.5))

im = ax.imshow(cm, cmap="Blues")
ax.set_title(f"Confusion Matrix — Logistic Regression (threshold = {threshold:.2f})", pad=14, fontsize=12)

# Ticks & labels
ax.set_xticks([0,1]); ax.set_yticks([0,1])
ax.set_xticklabels(["Pred: Non-Fraud (0)", "Pred: Fraud (1)"], fontsize=10)
ax.set_yticklabels(["Actual: Non-Fraud (0)", "Actual: Fraud (1)"], fontsize=10)

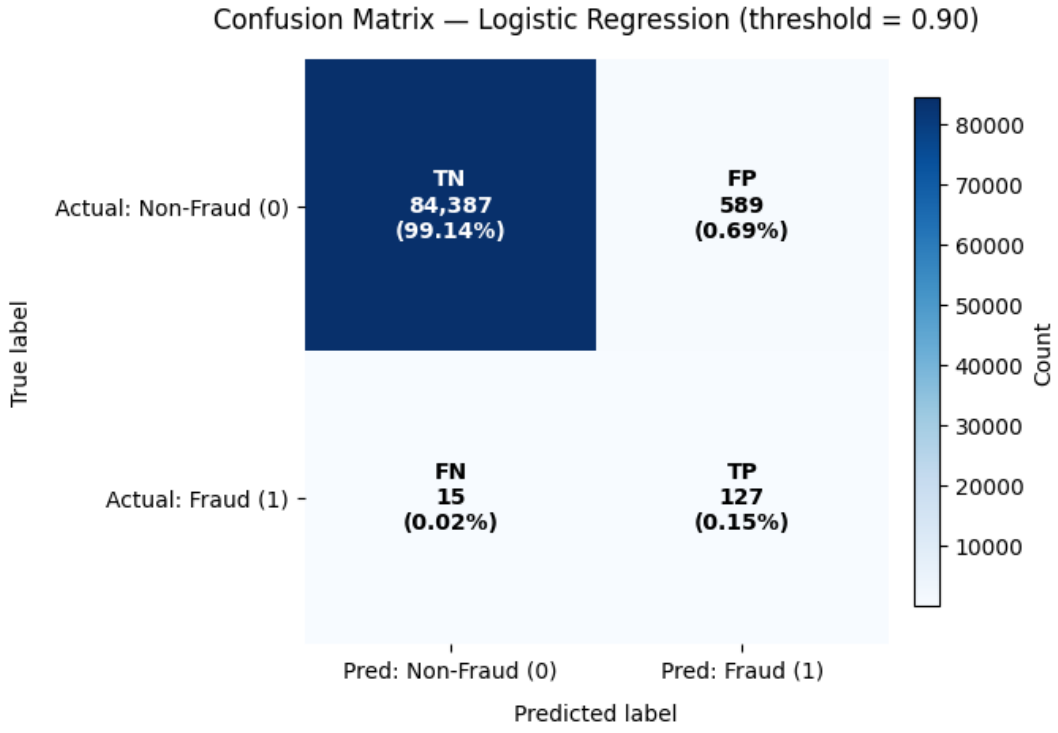
# Cell annotations: count + percentage

for i in range(2):
    for j in range(2):
        count = cm[i, j]
        pct = 100.0 * count / total
        ax.text(j, i,
                f"{labels[i,j]}\n{count:,}\n({pct:.2f}%)",
                va="center", ha="center", fontsize=10, fontweight="bold",
                color="white" if (i == 0 and j == 0) else "black")

# Colorbar
cbar = fig.colorbar(im, ax=ax, fraction=0.04, pad=0.04)
cbar.ax.set_ylabel("Count", rotation=90)

# Cosmetics
ax.spines[:].set_visible(False)
ax.set_xlabel("Predicted label", labelpad=10)
ax.set_ylabel("True label", labelpad=10)
plt.tight_layout()
plt.show()

print("\nCost Assumptions:")
print(f"  • False alarm (FP) cost = {cost_fp}")
print(f"  • Missed fraud (FN) cost = {cost_fn}")
print(f"Total Cost = {cost_fp}*FP + {cost_fn}*FN = {cost_fp}*{fp:,} + {cost_fn}*{fn:,} = {total_cost:,}")
```



OUTPUT

```
Cost Assumptions:
  • False alarm (FP) cost = 2
  • Missed fraud (FN) cost = 5
Total Cost = 2*FP + 5*FN = 2*589 + 5*15 = 1,253
```

At the threshold of 0.9, the model achieved a **Recall of 0.8944**, coupled with the **Confusion Matrix** meaning it correctly detected 127 out of 142 fraudulent cases, while only missing 15. This high recall reflects the effect of class weighting, which prioritizes minimizing false negatives in fraud detection. However, the **Precision is just around 18%**, indicating that approximately 18% of the transactions flagged as fraud were actually fraudulent. In practice, the huge difference between **Precision** and **recall** indicate that the model is highly effective at catching fraud, though it also generates a large number of false alarms that would require manual investigation. The resulting **F1-score of 0.2960** highlights the imbalance between Precision and Recall.

From the **Confusion Matrix (TN = 84,387; FP = 589; FN = 15; TP = 127)**, we see that 82,587 of legitimate transactions are classified correctly, but the number of false positives is substantial, at 589. This trade-off illustrates the central challenge in fraud detection: **maximizing recall to capture rare but costly fraud cases while maintaining sufficient precision to avoid excessive investigation costs**. To encouter this challenge, we decide to expand our research on more advanced machine LightGBM and XGBoosts.

2) Light Gradient Boosting Model (LightGBM)

The decision to use LightGBM and XGBoosts is closely related to the mathematical algorithm behind them, which is based on **Taylor's Theorem**, first introduced by English mathematician **Brook Taylor** in 1715. This theorem provides an approximation of a non-linear function using the **Gradient (first derivative)** and **Hessian (second derivative)** (*Taylor, 1715*). In modern machine learning, this concept is operationalized through the **second-order Taylor series expansion**, which is embedded in the optimization processes of both LightGBM and XGBoost between 2016 and 2017. These techniques are implemented in code by **Microsoft Research Asia** (LightGBM) and **Tianqi Chen** (XGBoost), respectively.

Specifically, LightGBM and XGBoost utilize exactly both linear approximation and second-order Taylor expansion to optimize the objective function efficiently.

Linear Approximation:

$$f(x) \approx f(\bar{x}) + \nabla f(\bar{x})^T (x - \bar{x})$$

Second-order Taylor Expansion:

$$f(x) = [f(\bar{x}) + \nabla f(\bar{x})^T (x - \bar{x})] + [\frac{1}{2}(x - \bar{x})^T \nabla^2 f(\hat{x})(x - \bar{x})]$$

In terms of Fraud Transaction Detection, once our objective function, derived from Amount, Time, and other 28 PCA-transformed variables, which is obviously non-linear. Therefore, it is plausible for applying Taylor's Theorem through LightGBM and XGBoost as the models might potentially yield an outstanding results by capturing complex patterns in the data.

PYTHON CELL 27

```python
# 1) Grid to search
max_depth_values = list(range(1, 11))   # 1..10
nrounds_values   = list(range(1, 31))   # 1..30
AUC_matrix_lgb = np.full((len(max_depth_values), len(nrounds_values)), np.nan)

# 2) 5-fold Stratified CV
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2025)

# 3) CV loop
for i, md in enumerate(max_depth_values):
    for j, nr in enumerate(nrounds_values):
        fold_scores = []
        for tr_idx, val_idx in cv.split(x_train, y_train):
            X_tr, X_val = x_train.iloc[tr_idx], x_train.iloc[val_idx]
            y_tr, y_val = y_train.iloc[tr_idx], y_train.iloc[val_idx]

            # imbalance weight per fold
            pos = int(y_tr.sum()); neg = len(y_tr) - pos
            spw = neg / pos

            clf = lgb.LGBMClassifier(
                max_depth=md,
                n_estimators=nr,
                learning_rate=0.1,
                subsample=0.9,
                colsample_bytree=0.9,
                scale_pos_weight=spw,
                random_state=2025,
                n_jobs=-1,
                verbose=-1
            )
            clf.fit(X_tr, y_tr)
            y_val_prob = clf.predict_proba(X_val)[:, 1]
            fold_scores.append(roc_auc_score(y_val, y_val_prob))

        AUC_matrix_lgb[i, j] = float(np.mean(fold_scores))
```

We evaluate LightGBM by implementing a **5-fold cross-validation** loop over a grid of hyperparameters. The grid search spans **tree depth** (max_depth, ranging from 1 to 10) and number of **boosting rounds** (n_estimators, ranging from 1 to 30). This grid search method allows us to study how model complexity (via depth) and number of iterations interact to affect predictive performance. In each iteration of cross-validation, the training data is seperated into five stratified folds, ensuring that the fraud-to-non-fraud ratio is preserved in both training and validation subsets. For a given parameter setting, four folds are used for model training while the fifth fold serves as a validation set, and this process is repeated until every fold has served as validation, and the performance metrics are averaged across the five runs.

Due to the severe class imbalance in the dataset, it is essential to apply class weighting to prevent fraudulent transactions from being overshadowed by the majority of non-fraudulent cases. At the same time, the weighting must ensure that misclassifying a fraud case carries a substantially higher penalty than misclassifying a legitimate transaction, thereby improving the model's sensitivity to the minority class. To achieve this, within each cross-validation fold, we recalculate the imbalance ratio as:

$$\text{scale_weight} = \frac{\text{number of negatives}}{\text{number of positives}} = \frac{284{,}315}{492} = 577.69$$

and this formula also resulted in 578 times scale-up for fraud cases and passed into the following steps in plotting LightGBM model.

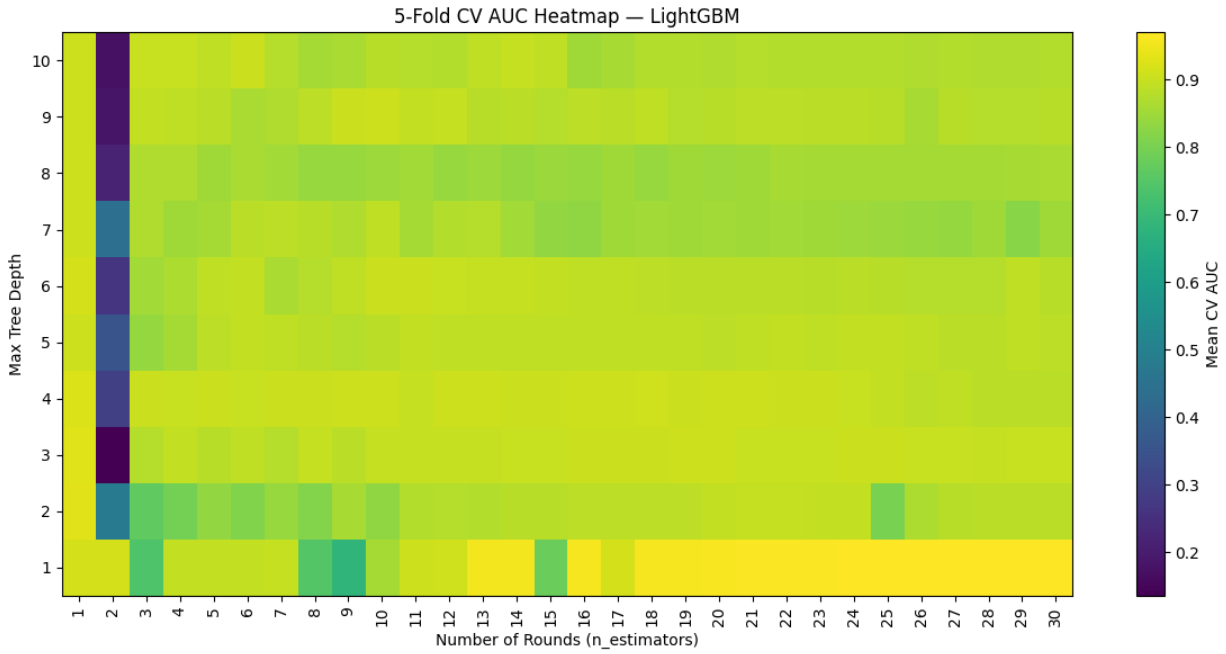
● PYTHON CELL 29

```
# Heatmap
plt.figure(figsize=(12, 6))
im = plt.imshow(AUC_matrix_lgb, aspect="auto", origin="lower", cmap="viridis")
cbar = plt.colorbar(im)
cbar.set_label("Mean CV AUC")

# ticks & labels
plt.xticks(ticks=np.arange(len(nrounds_values)), labels=nrounds_values, rotation=90)
plt.yticks(ticks=np.arange(len(max_depth_values)), labels=max_depth_values)
plt.xlabel("Number of Rounds (n_estimators)")
plt.ylabel("Max Tree Depth")
plt.title("5-Fold CV AUC Heatmap — LightGBM")

plt.tight_layout()
plt.show()

# Best params by CV AUC
best_i, best_j = np.unravel_index(np.nanargmax(AUC_matrix_lgb), AUC_matrix_lgb.shape)
best_auc = AUC_matrix_lgb[best_i, best_j]
best_depth = max_depth_values[best_i]
best_rounds = nrounds_values[best_j]
print(f"Best CV AUC (LightGBM) = {best_auc:.4f} with max_depth={best_depth}, n_estimators={best_rounds}")
```



OUTPUT

```
Best CV AUC (LightGBM) = 0.9695 with max_depth=1, n_estimators=29
```

The LightGBM heatmap highlights the algorithm of its modeling method, which uses a second-order Taylor expansion, and **majorly focuses on the Gradient** to minimize the computational time, while effectively updates to the model (*Ke et al., 2017*). By leveraging histogram-based algorithms and a leaf-wise growth strategy, LightGBM is able to significantly reduce computational time and maintaining high predictive accuracy (*Ke et al., 2017*).

From the cross-validation results, the best parameter setting was **max_depth = 1** with **n_estimators = 29**, yielding a **mean CV AUC of 0.9695**. This outcome suggests that the model effectively combines many simple, single-split trees across boosting rounds to achieve strong predictive power. However, as the mean CV AUC of LightGBM is lower than Logistic Regression, we hypothesize that LightGBM's structure may be too simple to

fully capture the complexity of fraud detection in an imbalanced dataset. From our perspective, LightGBM's reliance on repeating weak learners might limit its robustness compared to models that incorporate explicit regularization mechanisms.

In other words, Logistic Regression directly applies regularization and penalty terms that adjust for class imbalance and stabilize coefficient estimates. This leads us to assume that Logistic Regression could provide a more stable and interpretable benchmark, whereas LightGBM's oversimplified configuration might underperform once evaluated more critically.

PYTHON CELL 31

```python
# Refit on full TRAIN with best params, TEST once

pos = int(y_train.sum())
neg = len(y_train) - pos
spw = neg / pos

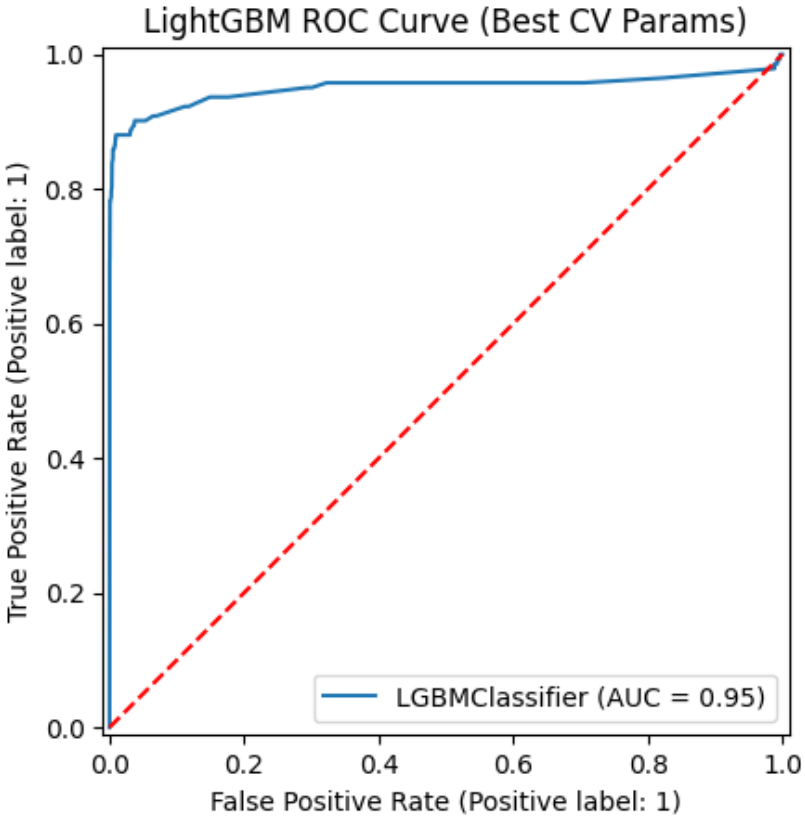
best_lgb = lgb.LGBMClassifier(
    max_depth=best_depth,
    n_estimators=best_rounds,
    learning_rate=0.1,
    subsample=0.9,
    colsample_bytree=0.9,
    scale_pos_weight=spw,
    random_state=2025,
    n_jobs=-1,
    verbose=-1
)
best_lgb.fit(x_train, y_train)

# Predictions on test
y_prob_test = best_lgb.predict_proba(x_test)[:, 1]
threshold = 0.9
y_pred_test = (y_prob_test >= threshold).astype(int)

# Precision, Recall, F1, AUC
precision = precision_score(y_test, y_pred_test, zero_division=0)
recall    = recall_score(y_test, y_pred_test, zero_division=0)
f1        = f1_score(y_test, y_pred_test, zero_division=0)
auc       = roc_auc_score(y_test, y_prob_test)

# ROC curve
RocCurveDisplay.from_estimator(best_lgb, x_test, y_test)
plt.plot([0, 1], [0, 1], "r--")
plt.title("LightGBM ROC Curve (Best CV Params)")
plt.show()

print(f"AUC (prob-based): {auc:.4f}")
print("\nClassification report:")
print(classification_report(y_test, y_pred_test, digits=4))
```



OUTPUT

```
AUC (prob-based): 0.9506

Classification report:
              precision    recall  f1-score   support

           0     0.9998    0.9773    0.9884     84976
           1     0.0609    0.8803    0.1140       142

    accuracy                         0.9772     85118
   macro avg     0.5304    0.9288    0.5512     85118
weighted avg     0.9982    0.9772    0.9870     85118
```

To validate our earlier hypothesis, we fitted the LightGBM model using its fine-tuned hyperparameters on the full training set and assessed its performance on the test set. The ROC curve with the best cross-validation parameters reported an **AUC of 95.06%**, which is slightly lower than the AUC achieved by the fine-tuned Logistic Regression model. In addition, the **Recall (0.8803)**, **Precision (0.0609)**, and **F1-score (0.1140)** are all lower compared to Logistic Regression at the same **threshold = 0.9**, meaning that the ability to detect fraudulent of LightGBM is weaker (lower Recall), while it raise significant amount of false alarms (lower Precision).

● PYTHON CELL 33

```python
# --- Confusion Matrix & Cost ---
cm = confusion_matrix(y_test, y_pred_test)
tn, fp, fn, tp = cm.ravel()
total = cm.sum()

C_FP = 2   # false alarm cost
C_FN = 5   # missed fraud cost
total_cost = C_FP*fp + C_FN*fn

# --- Pretty Confusion Matrix (matplotlib only) ---
labels = np.array([["TN", "FP"], ["FN", "TP"]])
fig, ax = plt.subplots(figsize=(6.5, 5.5))
```

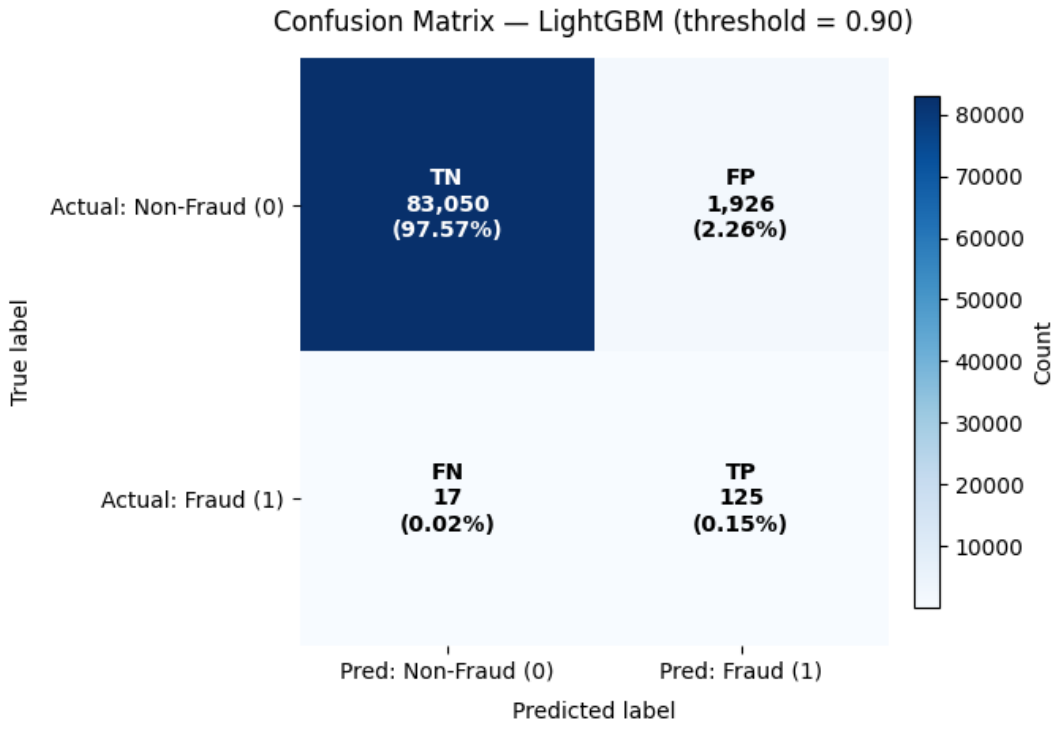
```
im = ax.imshow(cm, cmap="Blues")
ax.set_title(f"Confusion Matrix — LightGBM (threshold = {threshold:.2f})", pad=12, fontsize=12)

ax.set_xticks([0,1]); ax.set_yticks([0,1])
ax.set_xticklabels(["Pred: Non-Fraud (0)", "Pred: Fraud (1)"], fontsize=10)
ax.set_yticklabels(["Actual: Non-Fraud (0)", "Actual: Fraud (1)"], fontsize=10)

for i in range(2):
    for j in range(2):
        count = cm[i, j]
        pct = 100.0 * count / total
        ax.text(j, i, f"{labels[i,j]}\n{count:,}\n({pct:.2f}%)",
                va="center", ha="center", fontsize=10, fontweight="bold",
               color="white" if (i == 0 and j == 0) else "black")

cbar = fig.colorbar(im, ax=ax, fraction=0.04, pad=0.04)
cbar.ax.set_ylabel("Count", rotation=90)
ax.spines[:].set_visible(False)
ax.set_xlabel("Predicted label", labelpad=8)
ax.set_ylabel("True label", labelpad=8)
plt.tight_layout()
plt.show()

print("\nCost Assumptions:")
print(f"  • False alarm (FP) cost = {C_FP}")
print(f"  • Missed fraud (FN) cost = {C_FN}")
print(f"Total Cost = {C_FP}*FP + {C_FN}*FN = {C_FP}*{fp:,} + {C_FN}*{fn:,} = {total_cost:,}")
```



OUTPUT

```
Cost Assumptions:
  • False alarm (FP) cost = 2
  • Missed fraud (FN) cost = 5
Total Cost = 2*FP + 5*FN = 2*1,926 + 5*17 = 3,937
```

By evaluating the results of **Precision**, **Recall**, and **F1-Score**, and transforming it into **Confusion Matrix**, we can identify that while LightGBM successfully detected most fraud cases (125 out of 142), it also generated a higher number of **false positives (1,926)**, leading to more false alarms. More importantly, **17 fraud cases went undetected**, which highlights a weaker detection ability compared to Logistic Regression. As we incorporating

the cost framework, where false negatives are penalized five times more heavily and false positives twice as much as correct non-fraud predictions, the model generates an additional total of **3,937 cost units**. These findings reinforce our earlier assumption: LightGBM's reliance on shallow splits in this configuration limits its robustness on an imbalanced fraud dataset, making it less effective than **Logistic Regression (raises 1,253 additional cost units)** despite its computational efficiency.

Throughout the LightGBM model, we have evidence to conclude that although advanced models may perform efficiently and effectively in some prediction tasks, working with imbalanced data requires more than just model complexity. Specifically, tuning techniques and methodological choices play a critical role in fraud detection for the financial and banking industry. In this context, the use of regularization (C) and penalty shrinkage (Ridge and Lasso) in Logistic Regression may outweigh LightGBM's reliance on repeated single-split trees across many boosting rounds.

In comparison, prior studies analyzing the same Worldline–ULB fraud detection dataset have noted that the distribution of the transaction amount is highly right-skewed, with fraudulent transactions generally concentrated at higher amounts compared to non-fraudulent ones (*Chang et al., 2024*). This finding supports our assumption that the probability of fraud increases respects to transaction amount, even if this relationship is gradual rather than characterized by sharp jumps. Particularly, fraud does not suddenly appear at a specific threshold of transaction size, but it tends to rise steadily with the amount. For this reason, the fraud cases might be better captured by a **Logistic Regression** model, which directly estimates the probability of fraud as a continuous function of the predictors. In contrast, tree-based models like **LightGBM** attempt to approximate these relationships using a series of simple model splits, which is useful in situations where discontinuities exist, but less effective when the underlying process is continuous.

3) Extreme Gradient Boosting Model (XGBoost)

Given LightGBM's reliance on shallow trees and its limitations in capturing continuous relationships within our dataset, we turned to XGBoost as an alternative boosting method. Like LightGBM, XGBoost employs a Second-Order Taylor Expansion that uses both the Gradient (first derivative) and the Hessian (second derivative) to optimize the objective function (*Chen & Guestrin, 2016*). Its key distinction is the depth-wise growth strategy, which **expands trees level by level to ensure a more balanced structure**. This approach enhances stability and reduces the risk of overfitting, though it is generally more computationally intensive compared to LightGBM (*Chen & Guestrin, 2016*). However, XGBoost remains a strong choice for fraud detection tasks, as it can effectively handle complex data structures—such as PCA-transformed predictors—while delivering robust and accurate predictions.

● PYTHON CELL 36

```python
# 1) Grid to search
max_depth_values = list(range(1, 11))   # 1..10
nrounds_values   = list(range(1, 31))   # 1..30

AUC_matrix = np.full((len(max_depth_values), len(nrounds_values)), np.nan)

# 2) 5-fold CV setup
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2025)

# 3) CV loop over grid (NO test set used here)
for i, md in enumerate(max_depth_values):
    for j, nr in enumerate(nrounds_values):
        fold_scores = []
        for tr_idx, val_idx in cv.split(x_train, y_train):
            X_tr, X_val = x_train.iloc[tr_idx], x_train.iloc[val_idx]
            y_tr, y_val = y_train.iloc[tr_idx], y_train.iloc[val_idx]

            # imbalance weight computed per-fold
            pos = int(y_tr.sum())
            neg = len(y_tr) - pos
            spw = neg / pos
```

```
            clf = xgb.XGBClassifier(
                max_depth=md,
                n_estimators=nr,
                learning_rate=0.1,
                subsample=0.9,
                colsample_bytree=0.9,
                scale_pos_weight=spw,
                eval_metric="auc",
                use_label_encoder=False,
                random_state=2025,
                verbosity=0
            )
            clf.fit(X_tr, y_tr)
            y_val_prob = clf.predict_proba(X_val)[:, 1]
            fold_scores.append(roc_auc_score(y_val, y_val_prob))

        AUC_matrix[i, j] = float(np.mean(fold_scores))
```

We perform a **5-fold stratified cross-validation** to tune XGBoost across a grid of hyperparameters, varying the maximum **tree depth from 1 to 10** and the number of **boosting rounds (estimators) from 1 to 30**. Following the same procedure as with LightGBM, we compute the mean cross-validated AUC for each parameter combination and record the results in an AUC matrix. This matrix is then visualized as a heatmap, allowing us to identify the regions of best performance and assess the stability of XGBoost across different configurations.

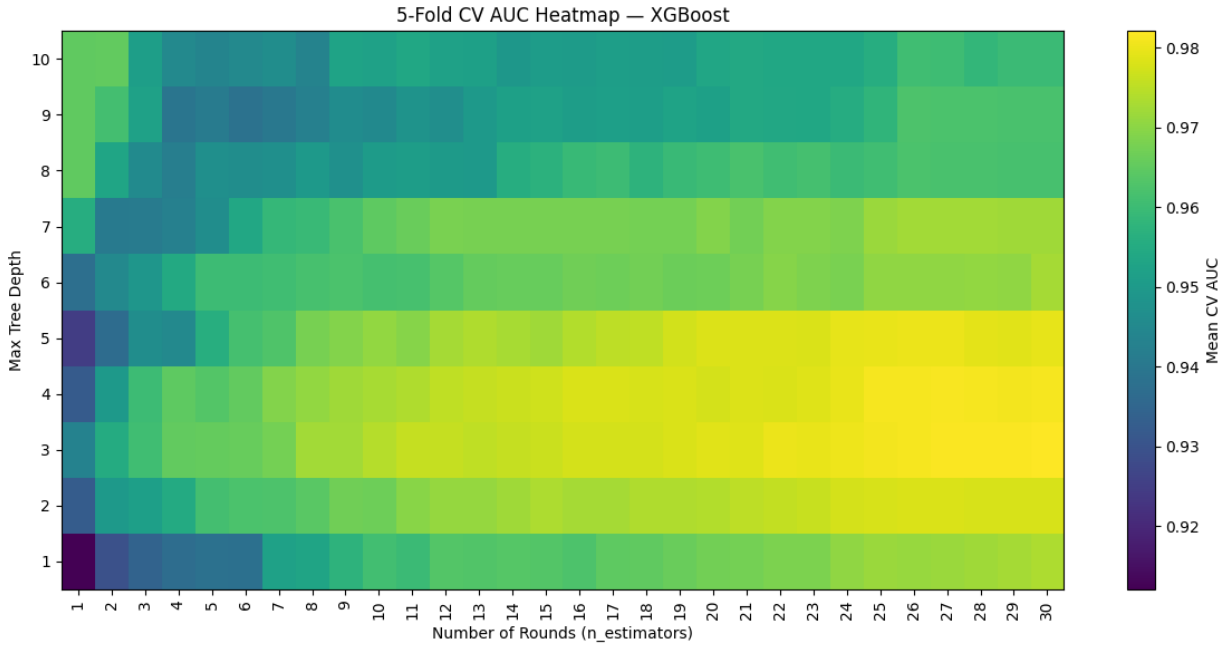
● PYTHON CELL 38

```python
# 4) heatmap
plt.figure(figsize=(12, 6))
im = plt.imshow(AUC_matrix, aspect="auto", origin="lower", cmap="viridis")
cbar = plt.colorbar(im)
cbar.set_label("Mean CV AUC")

plt.xticks(ticks=np.arange(len(nrounds_values)), labels=nrounds_values, rotation=90)
plt.yticks(ticks=np.arange(len(max_depth_values)), labels=max_depth_values)
plt.xlabel("Number of Rounds (n_estimators)")
plt.ylabel("Max Tree Depth")
plt.title("5-Fold CV AUC Heatmap — XGBoost")

plt.tight_layout()
plt.show()

# 5) best params by CV AUC
best_i, best_j = np.unravel_index(np.nanargmax(AUC_matrix), AUC_matrix.shape)
best_auc_cv = AUC_matrix[best_i, best_j]
best_depth = max_depth_values[best_i]
best_rounds = nrounds_values[best_j]
print(f"Best CV AUC = {best_auc_cv:.4f} with max_depth={best_depth}, n_estimators={best_rounds}")
```



OUTPUT

```
Best CV AUC = 0.9821 with max_depth=3, n_estimators=30
```

Within a grid of **10 tree depth levels and 30 boosting rounds**, the heatmap shows that XGBoost gradually improves as both parameters increase, with mean **CV AUC scores reaching above 0.98** in the best regions. Unlike LightGBM, which reached its best performance with very shallow trees, **XGBoost benefits from moderately deep trees (depth ≈ 3–6) combined with enough boosting rounds, reflecting its depth-wise growth strategy**. This pattern highlights XGBoost's ability to build stronger learners from the early stages and refine them through iterative boosting, eventually narrowing in on the best-performing configuration.

From the grid search, the best-performing XGBoost setting achieved a **mean CV AUC of 0.9821** with **max_depth = 3** and **n_estimators = 30**. This shows that moderately deep trees with sufficient boosting rounds provide the right balance between capturing complex patterns, while maintaining generalization. Compared to LightGBM, which was best with shallow single-split trees, XGBoost's depth-wise growth makes better use of additional complexity. The high AUC also confirms XGBoost's strong ability to separate fraud from non-fraud cases across thresholds, while the stability in nearby parameter settings reinforces the robustness of this configuration. Overall, these results highlight that XGBoost is effective in handling the imbalanced fraud dataset, especially when it grows beyond single-split trees without going too deep and risking overfitting.

● PYTHON CELL 40

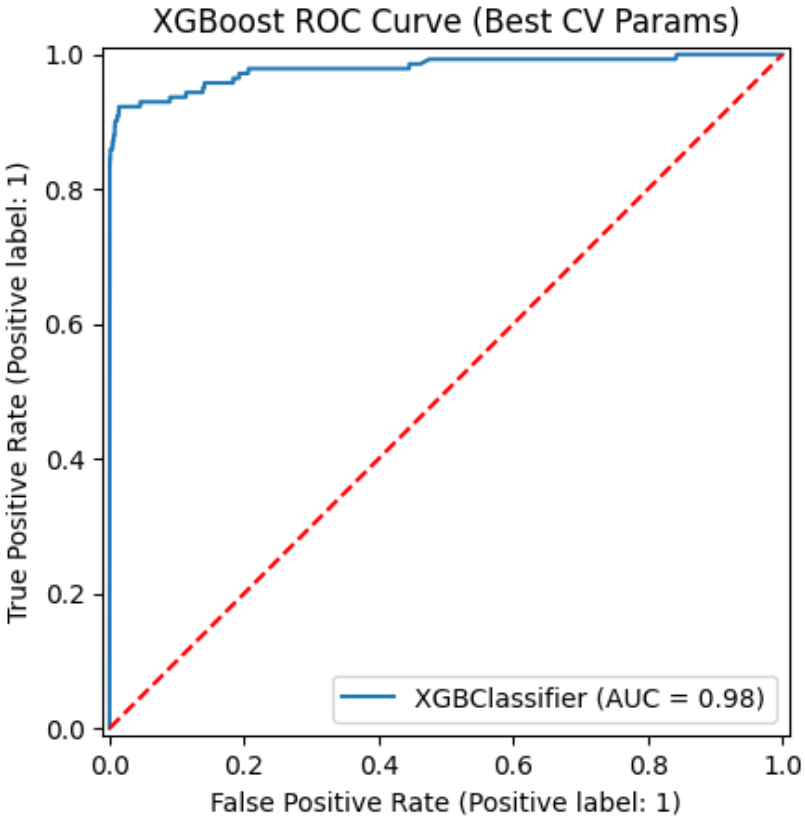
```python
# Refit on full TRAIN with best params, then evaluate ONCE on TEST
pos = int(y_train.sum())
neg = len(y_train) - pos
spw = neg / pos
best_xgb = xgb.XGBClassifier(
    max_depth=best_depth,
    n_estimators=best_rounds,
    learning_rate=0.1,
    subsample=0.9,
    colsample_bytree=0.9,
    scale_pos_weight=spw,
    eval_metric="auc",
    use_label_encoder=False,
    random_state=2025,
    verbosity=0
)
best_xgb.fit(x_train, y_train)
```

```python
# Predictions on test
y_prob_test = best_xgb.predict_proba(x_test)[:, 1]
threshold = 0.9
y_pred_test = (y_prob_test >= threshold).astype(int)

# Precision, Recall, F1, AUC
precision = precision_score(y_test, y_pred_test, zero_division=0)
recall    = recall_score(y_test, y_pred_test, zero_division=0)
f1        = f1_score(y_test, y_pred_test, zero_division=0)
auc       = roc_auc_score(y_test, y_prob_test)

# ROC curve
RocCurveDisplay.from_estimator(best_xgb, x_test, y_test)
plt.plot([0, 1], [0, 1], "r--")
plt.title(f"XGBoost ROC Curve (Best CV Params)")
plt.show()

print(f"AUC (prob-based): {auc:.4f}")
print("\nClassification report:")
print(classification_report(y_test, y_pred_test, digits=4))
```



OUTPUT

```
AUC (prob-based): 0.9792

Classification report:
              precision    recall  f1-score   support

           0     0.9997    0.9992    0.9995     84976
           1     0.6432    0.8380    0.7278       142

    accuracy                         0.9990     85118
   macro avg     0.8215    0.9186    0.8637     85118
weighted avg     0.9991    0.9990    0.9990     85118
```

To further analyze the performance of XGBoost, we refit the model with the best-tuned hyperparameters and plotted the ROC Curve. The curve demonstrates outstanding performance with an **AUC of 97.92%**, confirming the model's strong ability to distinguish between fraud and non-fraud cases across **thresholds of 0.9**. While this AUC is only slightly higher than that of Logistic Regression, the classification metrics show a notable improvement: XGBoost achieved a **Precision of 0.6432**, **Recall of 0.8380**, and an **F1-score of 0.7278** for fraud cases. These metrics indicate that XGBoost not only maintains balance Precision and Recall far more effectively than Logistic Regression, which struggled with low Precision despite high Recall.

The improvement in Precision is particularly important in fraud detection, as it reduces the number of false alarms compared to Logistic Regression. At the same time, Recall remains high, ensuring that the majority of fraud cases are correctly detected. As a result, this balance yields a much higher F1-score, indicating that XGBoost provides a more robust and practical solution for detecting fraud in real-world imbalanced datasets.

● PYTHON CELL 42

```python
# --- Confusion Matrix & Cost ---
cm = confusion_matrix(y_test, y_pred_test)
tn, fp, fn, tp = cm.ravel()
total = cm.sum()

C_FP = 2   # false alarm cost
C_FN = 5   # missed fraud cost
total_cost = C_FP*fp + C_FN*fn

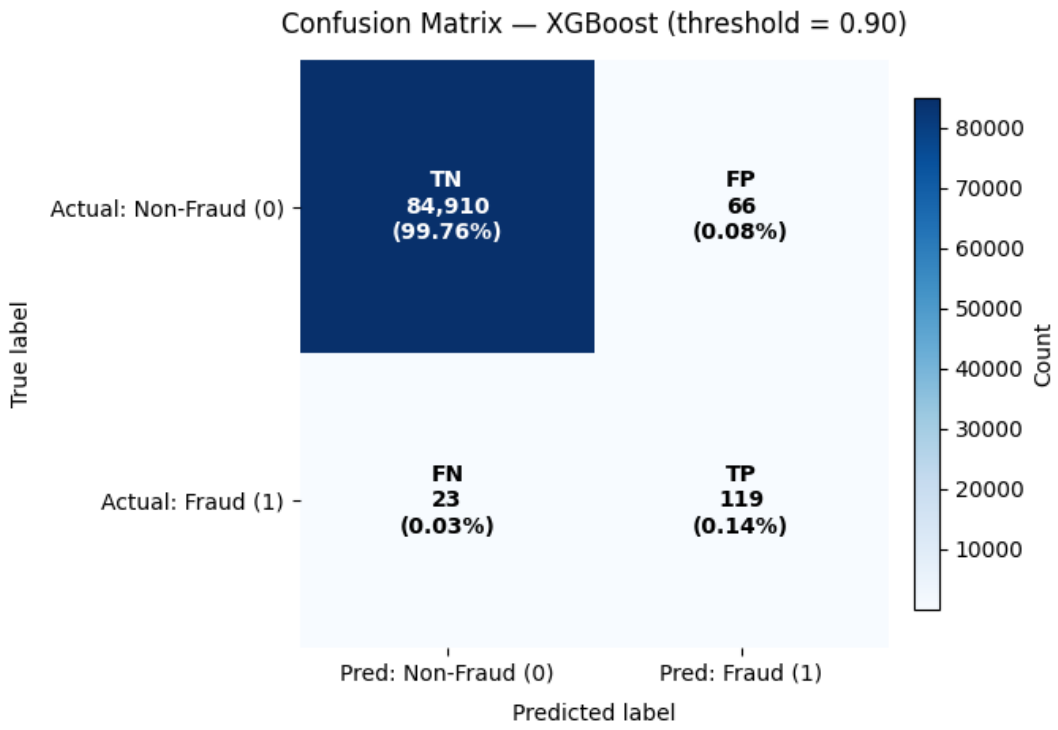
# --- Pretty Confusion Matrix (matplotlib only) ---
labels = np.array([["TN", "FP"], ["FN", "TP"]])
fig, ax = plt.subplots(figsize=(6.5, 5.5))
im = ax.imshow(cm, cmap="Blues")
ax.set_title(f"Confusion Matrix — XGBoost (threshold = {threshold:.2f})", pad=12, fontsize=12)

ax.set_xticks([0,1]); ax.set_yticks([0,1])
ax.set_xticklabels(["Pred: Non-Fraud (0)", "Pred: Fraud (1)"], fontsize=10)
ax.set_yticklabels(["Actual: Non-Fraud (0)", "Actual: Fraud (1)"], fontsize=10)

for i in range(2):
    for j in range(2):
        count = cm[i, j]
        pct = 100.0 * count / total
        ax.text(j, i, f"{labels[i,j]}\n{count:,}\n({pct:.2f}%)",
                va="center", ha="center", fontsize=10, fontweight="bold",
               color="white" if (i == 0 and j == 0) else "black")

cbar = fig.colorbar(im, ax=ax, fraction=0.04, pad=0.04)
cbar.ax.set_ylabel("Count", rotation=90)
ax.spines[:].set_visible(False)
ax.set_xlabel("Predicted label", labelpad=8)
ax.set_ylabel("True label", labelpad=8)
plt.tight_layout()
plt.show()

print("\nCost Assumptions:")
print(f"  • False alarm (FP) cost = {C_FP}")
print(f"  • Missed fraud (FN) cost = {C_FN}")
print(f"Total Cost = {C_FP}*FP + {C_FN}*FN = {C_FP}*{fp:,} + {C_FN}*{fn:,} = {total_cost:,}")
```



OUTPUT

```
Cost Assumptions:
  • False alarm (FP) cost = 2
  • Missed fraud (FN) cost = 5
Total Cost = 2*FP + 5*FN = 2*66 + 5*23 = 247
```

Building on the ROC analysis, we further evaluate **XGBoost's Confusion Matrix**. Out of 85,118 test transactions, the model correctly **identified 119 fraud cases** while missing 23. Although the number of missed fraud cases is roughly higher that of Logistic Regression and LightGBM (which detected 125–127 fraud cases), XGBoost drastically reduces false alarms, producing only **66 false positives** among nearly 85,000 non-fraud cases while still capturing the majority of fraud cases. This demonstrates a better trade-off between Precision and Recall, especially when we apply this **Confusion Matrix** on our cost framework (false alarms = 2×, missed frauds = 5×), the total cost is reduced to just **247 cost units**, which is dramatically lower than the costs associated with both Logistic Regression and LightGBM, . This result emphasizes that XGBoost not only delivers strong statistical performance but also provides a far more cost-effective and practical fraud detection solution in real-world financial settings. For these reasons, we conclude that XGBoost is the most advanced and effective model for fraud detection in our analysis, and we proceed to examine its feature importance to deliver valuable insights.

IV. Business Insights

PYTHON CELL 45

```python
# Get feature importance from the trained best_xgb model
importance = best_xgb.feature_importances_

# Build a DataFrame
importance_df = pd.DataFrame({
    "Feature": x_train.columns,
    "Importance": importance
})

# Sort by importance descending
```

```
importance_df = importance_df.sort_values(by="Importance", ascending=False).reset_index(drop=True)
importance_df.head(10).style.background_gradient(cmap="Blues")
```

RESULT TABLE

	Feature	Importance
0	V14	0.323806
1	V10	0.156703
2	V12	0.055975
3	V4	0.046028
4	V8	0.039609
5	V20	0.038020
6	V23	0.035982
7	V13	0.032479
8	Amount	0.028121
9	V21	0.027826

From the **Feature Importance table of XGBoost**, we can see that V14, V10, and V12 stand out as the top contributors in detecting fraudulent transactions. Among these, **V14** carries the largest weight with an **Importance of 0.3238**, followed by **V10 at 0.1567**, showing that these two components dominate the model's decision-making process. Since the dataset is PCA-transformed, we cannot directly interpret what each variable represents, but it is reasonable to assume that these components capture patterns related to unusual transaction behaviors, such as sharp deviations in transaction amount, timing irregularities, or combinations of features that are rare in normal activity but common in fraudulent cases. In comparison,

The next contributors, **V12** (0.056) and **V4** (0.046), also provide substantial explanatory power, suggesting that fraud detection does not rely on a single signal but rather a mix of several strong indicators. Interestingly, most of the other components such as V8, V20, V23, V13, and V21 show very low importance, meaning that the model finds less predictive value from them once the stronger features are accounted for.

An important assumption we can make here is that fraud cases tend to cluster around certain hidden patterns captured by these principal components (e.g., anomalies in spending velocity, correlation between transaction amount and merchant type, or sudden account behavior shifts). Thus, XGBoost's ability to prioritize V14, V10, V4, and V12 may reflect its strength in isolating the most relevant fraud signals from a large pool of mostly redundant or weak features. From a business perspective, these findings are valuable, especially when only a handful of features (V14, V10, V4, and V12) dominate the fraud detection process, suggesting that financial institutions can focus monitoring efforts on a smaller set of high-impact signals rather than spreading resources evenly across all variables. This not only improves the efficiency of fraud detection systems but also reduces computational costs in real-time monitoring, since the model does not need to rely on hundreds of weak or noisy signals.

V. Conclusion

Credit Card Fraud Detection remains one of the most critical challenges in the financial sector. In this study, we applied machine learning techniques to the **Worldline–ULB** fraud detection dataset, implementing and evaluating three models: Logistic Regression, Light Gradient Boosting (LightGBM), and Extreme Gradient Boosting (XGBoost). Each model was tuned using **5-fold cross-validation** with **regularization**, **penalty**, and **grid search** adjustments. Among them, XGBoost achieved the strongest performance, with an AUC score of **0.9792**

and consistently high Precision, Recall, and F1 scores. These results highlight the model's strong ability to detect fraudulent transactions, particularly through the importance it assigns to features such as V14 and V10, which likely capture outlier behaviors and irregular spending patterns.

Although the findings demonstrate the effectiveness of advanced boosting algorithms, the study is constrained by the dataset size of roughly **285,000 transactions**. Real-world financial institutions may operate on much larger and more complex datasets, where fraud patterns may also evolve dynamically. To strengthen robustness and applicability, future work should focus on real-time deployment, continuous retraining with updated data, and the integration of additional transaction features. By advancing in these directions, fraud detection systems can become more scalable, adaptive, and reliable, and ultimately minimizing financial losses while better protecting consumers against emerging fraud strategies.

References

1. Chang, V., Ali, B., Golightly, L., Ganatra, M. A., & Mohamed, M. (2024). Investigating Credit Card Payment Fraud with Detection Methods Using Advanced Machine Learning. Information, 15(8), 478. https://doi.org/10.3390/info15080478
2. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785
3. Cruz, B. (2025, January 27). 2025 *Credit Card Fraud Report: Trends and Prevention*. Security.org. https://www.security.org/digital-safety/credit-card-fraud-report/
4. Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). *Credit card fraud detection: A realistic modeling and a novel learning strategy. IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3784–3797. https://doi.org/10.1109/TNNLS.2017.2736643
5. Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441. https://doi.org/10.1037/h0071325
6. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). *LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems*, 30. https://papers.nips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf
7. Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 559–572. https://doi.org/10.1080/14786440109462720
8. Taylor, B. (1715). *Methodus incrementorum directa et inversa*. London: G. Strahan.